

في بايثون: من الفكرة إلى (OOP) المرجع الشامل للبرمجة كائنية التوجه ## الإتقان

أصلاً؟ OOP مقدمة: لماذا نحتاج إلى

تخيل أنك تبني كوخًا صغيرًا. يمكنك استخدام أدوات بسيطة وقطع خشب متناثرة لإنجاز المهمة. لكن ماذا لو طلب منك بناء مدينة كاملة؟ الطريقة العشوائية لن تنجح. ستحتاج إلى "مخططات بناء" لكل نوع من المباني (منازل، مستشفيات، مدارس)، وستحتاج إلى طريقة لتنظيم العلاقات بين هذه المباني.

هي ليست لغة برمجة، بل هي طريقة تفكير وأسلوب لتنظيم الكود (OOP). هذا هو بالضبط ما تفعله البرمجة كائنية التوجه. يسمح لنا ببناء برامج كبيرة ومعقدة عن طريق تجميع الأكواد في "كائنات" منظمة، تمامًا مثل مباني المدينة.

(Classes & Objects) الفصل الأول: مخطط البناء والمنتج الفعلي

1.1 object والكائن class الكلاس

- التي سيمتلکها (methods) والسلوكيات (attributes) إنه يصف الخصائص. هو **مخطط البناء (Class)** الكلاس. أي شيء يتم بناؤه منه. هو مجرد قالب نظري.
- هو **المنتج الفعلي** الذي يتم بناؤه من مخطط البناء. كل كائن هو نسخة مستقلة لها **(Object/Instance) الكائن**. بياناتها الخاصة.

Python

```
# class هو مخطط البناء للموظفين
class Employee:
    pass
```

```
# هما كائنان فعليان تم بناؤهما من المخطط emp1 و emp2
emp1 = Employee()
emp2 = Employee()
```

1.2 self وضمير المتكلم __init__ المصنع

- هي دالة "المصنع" أو "خط التجميع". يتم استدعاؤها تلقائيًا في كل مرة ننشئ فيها كائنًا جديدًا. **__init__**. وظيفتها هي إعداد الكائن وتعيين خصائصه الأولية.
- self.name = name** هو ضمير المتكلم الذي يشير إلى "الكائن الذي يتم بناؤه حاليًا". عندما تكتب **self**: "فأنت تقول: "في هذا الكائن المحدد الذي أبيضه الآن، اجعل خاصية الاسم تساوي القيمة التي استلمتها".

1.3 (Methods) والسلوكيات (Attributes) الخصائص

- Attributes:** هي البيانات أو المتغيرات التي تصف حالة الكائن (مثل name, price, color).
- Methods:** هي الدوال الموجودة داخل الكلاس والتي تصف الأفعال التي يمكن للكائن القيام بها.

Python

```
class Employee:
```

```

def __init__(self, fName, lName, salary):
    # Attributes
    self.fName = fName
    self.lName = lName
    self.salary = salary

# Method
def fullname(self):
    return "{} {}".format(self.fName , self.lName)

emp1 = Employee("Yousef","Ahmed", 5000)
print(emp1.fullname()) # استدعاء دالة (سلوك)

```

تطبع هذه `display_info`، ودالة `student_id` و `name` له خصائص `Student` أنشئ كلاسًا بسيطًا: **تمرين صغير** المعلومات.

الفصل الثاني: البيانات المشتركة والأعمدة الأربعة

2.1 (Class Attributes) متغيرات الكلاس

هي خصائص مشتركة تنتمي إلى "مخطط البناء" نفسه وتشاركها جميع الكائنات. إذا تغيرت، فإنها تتغير للجميع. فكر فيها كـ "مذكرة داخلية" تُعمم على كل موظفي الشركة.

Python

```

class Employee:
    raise_amount = 1.04 # متغير كلاس

    def apply_raise(self):
        self.salary = int( self.salary * self.raise_amount ) # يمكن الوصول
        self له عبر

```

2.2 OOP الأعمدة الأربعة

1. **التغليف (Encapsulation):** آلية تنفيذه هي عبر جعل المتغيرات "خاصة" والتحكم في الوصول إليها.
 - **عام (Public):** `self.name` يمكن الوصول له مباشرة مثل.
 - **محمي (Protected):** `self._name` يبدأ بـ `_` استخدام خارج الكلاس.
 - **خاص (Private):** `self.__name` يبدأ بـ `__` ليصعب الوصول إليه، وهو يستخدم لمنع التضارب في الوراثة (Mangling).
2. **التجريد (Abstraction):** إخفاء التفاصيل المعقدة خلف واجهة بسيطة. أنت تستخدم `emp1.apply_raise()` دون الحاجة لمعرفة كيف تتم العملية الحسابية بالضبط.
3. **الوراثة (Inheritance):** قدرة كلاس جديد (ابن) على أن يرث من كلاس موجود (أب).
4. **تعددية الأشكال (Polymorphism):** قدرة كائنات مختلفة على الاستجابة لنفس استدعاء الدالة كل بطريقة الخاصة.

يرثان منه ويعيدان تعريف Cat و Dog ثم أنشئ كلاس ابن make_sound ودالته Animal أنشئ كلاس: تمرين صغير
ليصدرا صوتهما الخاص make_sound.

الفصل الثالث: الوراثة المتقدمة والأدوات الخاصة

3.1 التواصل مع الأب: super()

ضرورية لتجنب تكرار الكود، و super().__init__() هي أداة ذكية تسمح للابن باستدعاء دوال الأب
super().some_method() تسمح لنا بتوسيع سلوك الأب

Python

```
class Developer(Employee):
    def __init__(self, fName, lName, phone, salary, prog_lang):
        super().__init__(fName, lName, phone, salary) # استدعاء مصنع الأب
        self.prog_lang = prog_lang
```

3.2 @classmethod و @staticmethod

- @classmethod: لإنشاء كائنات "مصنع بديل" أفضل استخدام لها هو cls. (دالة خاصة بالكلاس) تستقبل من بيانات غير قياسية
- @staticmethod: دالة مساعدة عامة لا تحتاج للكلاس أو الكائن

اسمه @classmethod ثم أنشئ model. و make يستقبل __init__ اجعل Car. أنشئ كلاس: تمرين صغير
وينشئ منه كائن سيارة "Toyota-Camry" يستقبل نصًا from_string

الفصل الرابع: جعل الكائنات تتصرف كأنها مدمجة

4.1 __str__ و __repr__

- __str__: مثل واجهة السيارة. (print(obj)) للطباعة سهلة القراءة للمستخدم
- __repr__: للطباعة الفنية للمطور، وقاعدتها الذهبية هي أن تُرجع كودًا صالحًا لإنشاء الكائن مرة أخرى. مثل تقرير الميكانيكي

4.2 Operator Overloading (__add__, __len__)

تعمل مع كائناتك len() و + جعل المعاملات مثل

Python

```
# داخل كلاس Employee
def __add__(self, other):
    return self.salary + other.salary
```

4.3 الخصائص (Properties)

(Getters/Setters) هي الطريقة الاحترافية للتحكم في الوصول للمتغيرات

- يتم تنفيذه عند قراءة قيمة المتغير **Getter (@property)** الـ.
- يتم تنفيذه عند تعيين قيمة جديدة، ويسمح بالتحقق من صحتها **Setter (@*.setter)** الـ.
- بينما يبقى اسم الخاصية عامًا (`self._name`) لاسم المتغير الحقيقي لتجنبها، نستخدم: **مشكلة الحلقة اللانهائية** (`self.name`).

يتأكد من أن السعر لا يمكن أن يكون سالبًا Setter اجعل الـ `price` بخاصية `Product` أنشئ كلاس: تمرين صغير

Properties , Getter , Setter , Deleter

مراجعة شاملة: Getters, Setters, Deleters

1. الـ Getter (الحصول على القيمة)

كوري شيفر شرحها لكنه استخدم **Getter!** هي الـ `@property` الدالة التي تضع فوقها. هذا هو الجزء الذي سألت عنه لوصفها "property" مصطلح

- ما هو؟ هو الكود الذي يتم تنفيذه تلقائيًا عندما تحاول قراءة قيمة الخاصية.
- الكود:

Python

```
@property
def fullname(self):
    # هذا هو الـ Getter
    print("Getting fullname!")
    return f"{self.fName} {self.lName}"
```

- (ينفذ الكود أعلاه) `print(empl.fullname)`: الاستدعاء

2. الـ Setter (تعيين قيمة جديدة)

- ما هو؟ هو الكود الذي يتم تنفيذه تلقائيًا عندما تحاول تعيين قيمة جديدة للخاصية باستخدام علامة.
- الكود:

Python

```
@fullname.setter
def fullname(self, name):
    # هذا هو الـ Setter
    print("Setting fullname!")
    fName, lName = name.split(' ')
```

```
self.fName = fName
self.lName = lName
```

- **الاستدعاء:** `emp1.fullname = "Yousef Ahmed"` (ينفذ الكود أعلاه).
- **الفائدة:** `(self.fName, self.lName)` يسمح لك بالتحقق من صحة القيمة الجديدة أو، كما في هذا المثال، تحديث متغيرين آخرين: بناءً على قيمة واحدة.

3. حذف القيمة (Deleter الـ)

- على الخاصية `del` ما هو؟ هو الكود الذي يتم تنفيذه تلقائياً عندما تستخدم كلمة
- **الكود:**

Python

```
@fullname.deleter
def fullname(self):
    # هذا هو الـ Deleter
    print("Deleting fullname!")
    self.fName = None
    self.lName = None
```

- **الاستدعاء:** `del emp1.fullname` (ينفذ الكود أعلاه).

الخلاصة

`fullname` ثلاث مرات. أنت تعرف ثلاث عمليات منفصلة لخاصية واحدة اسمها `fullname`، إذاً، أنت لا تعرف الدالة

1. (Getter) ماذا تفعل عند قراءتها.
2. (Setter) ماذا تفعل عند الكتابة عليها.
3. (Deleter) ماذا تفعل عند حذفها.

الفائدة الكبرى من كل هذا هو أنه يمنحك تحكماً كاملاً في المتغيرات، بينما يظل المستخدم النهائي للكلاس يتعامل معها كأنها **(Abstraction)** هذا مثال رائع على التجريد. (`emp1.fullname`) متغيرات عادية وبسيطة

الفصل الخامس : Special Methods

Special Methods ما هي الـ

هذه `__init__` ببساطة، هي مجموعة من الدوال المعرفة مسبقاً في بايثون والتي تبدأ وتنتهي بشرطتين مزدوجتين (مثلاً `len()`، دالة `-`، الطرح `+`) الدوال تسمح للكائنات التي ننشئها بأن تتفاعل مع العمليات المدمجة في اللغة، مثل الجمع `print()` والطباعة.

فكر في هذه الدوال كأنها "أزرار" خاصة في الكائن الخاص بك. بايثون تعرف هذه الأزرار، وعندما تقوم **الأداة الذهنية** بعملية معينة، تبحث بايثون عن الزر المناسب في الكائن الخاص بك وتضغط عليه.

أهم الدوال الخاصة التي تمت مراجعتها

1. المصنع: `__init__(self, ...)`

هذه هي أشهر دالة خاصة، وقد أتقنتها بالفعل. هي "الزر" الذي تضغطه بايثون تلقائياً عند إنشاء كائن جديد.

2. بطاقة التعريف: `__str__(self)` و `__repr__(self)`

، لكن لكل منهما هدف مختلف (string) هاتان الدالتان مسؤولتان عن تحويل الكائن الخاص بك إلى نص.

- **`__repr__(self)` (Representation):**
 - إرجاع نص واضح وغير غامض يصف الكائن، وهو موجه للمبرمجين. القاعدة العامة هي أن هذا **الهدف** النص يجب أن يكون، إن أمكن، كود بايثون صالحاً يمكنه إعادة إنشاء الكائن.
 - أو "بطاقة المواصفات الفنية" للمنتج "الرقم التسلسلي" فكر فيه كأنه **الأداة الذهنية**.
 - مثال: `return f"Employee('{self.first}', '{self.last}', {self.pay})"`
- **`__str__(self)` (String):**
 - إرجاع نص سهل القراءة وجميل، وهو موجه للمستخدم النهائي. هذه هي الدالة التي تستدعيها **الهدف** تلقائياً `print()`.
 - على العكس "اسم المنتج ووصفه" فكر فيه كأنه **الأداة الذهنية**.
 - مثال: `return f'{self.fullname()} - {self.email}'`
- **ملاحظة:** وتستخدمها بدلاً منها `__repr__` ستبحث عن `print()` فإن لم تقم بتعريف `__str__` إذا لم تقم بتعريف `__repr__`.

Operator Overloading: جعل الكائنات تتصرف كأنها مدمجة

هنا يكمن قلب الموضوع الذي سألك عنه الدكتور

1. `__add__(self, other)`: + التحميل الزائد

- بين كائنين من الكلاس الخاص بك (+) تعريف ما يحدث عند استخدام علامة الجمع: **الهدف**
- قام بتعريفها لجمع رواتب الموظفين: **مثال كوري شيفر**

Python

```
def __add__(self, other):
    return self.pay + other.pay
```

- سيقوم الآن بجمع راتب الموظف الأول مع الثاني `print(emp1 + emp2)`: **الاستخدام**

2. `__len__(self)`: **التحميل الزائد len()**

- المدمجة على الكائن الخاص بك `len()` تعريف ما يحدث عندما تستدعي دالة: **الهدف**
- قام بتعريفها لترجع طول الاسم الكامل للموظف: **مثال كوري شيفر**

Python

```
def __len__(self):
    return len(self.fullname())
```

- سيعيد الآن عدد حروف الاسم الكامل للموظف `print(len(emp1))`: **الاستخدام**

الخلاصة:

الدوال الخاصة هي بوابتك لجعل الكلاسات التي تنشئها تتكامل بسلاسة مع لغة بايثون، وتتصرف تمامًا مثل الأنواع المدمجة (strings, lists, integers)، مما يجعل الكود الخاص بك أكثر وضوحًا وقوة.

OOP الفصل السادس: الغوص العميق في آليات ومبادئ

1. بناء الحصن المنيع (Encapsulation): التغليف

- في وحدة واحدة (الكلاس)، (Methods) والعمليات التي تجرى عليها (Attributes) هو تجميع البيانات: **المبدأ** والأهم من ذلك، **التحكم في الوصول** إلى هذه البيانات. الهدف هو حماية بيانات الكائن من التعديل العشوائي وغير المنطقي.
- **آليات التنفيذ في بايثون**:
 - يمكن الوصول إليه وتعديله من أي مكان. (`self.name` مثل) أي متغير لا يبدأ بـ **(Public) العام** (مثل ولجهة المحل).
 - هذا لا يمنع الوصول، (`self._balance` مثل) يبدأ بشرطة سفلية واحدة **(Protected) المحمي** بل هو "اتفاق شرف" بين المبرمجين يقول: "هذا متغير داخلي، لا يفترض بك تعديله مباشرة من خارج الكلاس (مثل غرفة الموظفين فقط)".
 - بايثون تقوم بإعادة تسميته داخليًا. (`self.__id` مثل) يبدأ بشرطتين سفليتين **(Private) الخاص** (مثل). ليصعب جدًا الوصول إليه. يستخدم لمنع التضارب العرضي في الوراثة (`ClassName__id`) (خزنة البنك).

2. قيادة السيارة دون فهم المحرك (Abstraction): التجريد

- إخفاء التفاصيل التنفيذية المعقدة خلف واجهة بسيطة وسهلة الاستخدام. الهدف هو تقليل التعقيد للمستخدم الذي يتعامل مع الكائن.
- المستخدم النهائي يتعامل مع دالة بسيطة جدًا اسمها `Inventory` عندما أنشأنا كلاس **مثال عملي**، هو لا يحتاج أن يعرف أن هذه الدالة في الخلفية تقوم بالمرور `inventory.total_inventory_value` لكل كائن، وتجمع النتائج. كل هذا التعقيد تم `total_value` على قاموس، وتصل إلى كل كائن، وتستدعي خاصية "تجريده" خلف واجهة بسيطة.

3. بصمة الابن الخاصة (Overriding): إعادة التعريف

- هو أن يقوم الكلاس الابن بتقديم تنفيذ خاص به لدالة ورثها من الأب. هذا يسمح للابن بتخصيص السلوك: **المبدأ** الموروثة.
- **أنواعه**:
 1. استبدلت `Lion` في `make_sound` الابن يتجاهل تمامًا ما يفعله الأب. (مثال: دالة: **الاستبدال الكامل** `Animal` بالكامل دالة).
 2. ثم يضيف عليه منطق `super().method()` الابن ينفذ منطق الأب أولاً باستخدام **التوسيع والإضافة** تم أضافت معلومات `super().info()` استدعت `ElitePlayer` في `info` الخاص. (مثال: دالة: **العنصر الخاص**).

4. نفس الزر، نتائج مختلفة (Polymorphism): تعددية الأشكال

- هو قدرة كائنات من كلاسات مختلفة على الاستجابة لنفس استدعاء الدالة، كل بطريقته الخاصة: **المبدأ**
- بايثون لا يهتم بنوع الكائن بقدر ما يهتم بما إذا كان الكائن **يستطيع تنفيذ (Duck Typing) كيف يعمل في بايثون** واستدعينا zoo_animals في مثال حديقة الحيوان، عندما مررنا على قائمة الدالة المطلوبة لديه دالة animal، بايثون لم تقل "هل هذا أسد؟"، بل قالت "هل هذا الشيء animal.make_sound()؟ إذا نعم، نفذها". هذا المبدأ يجعل الكود مرناً جداً make_sound

5. (Operator Overloading) وتحميل المعاملات (Special Methods) الدوال الخاصة

الدوال الخاصة هي الآلية التي تنفذ بها بايثون تحميل المعاملات. هذان المصطلحان مرتبطان بشكل وثيق

- يفهم كيفية التعامل len() أو > أو + مثل (Operator) "هو جعل "المعامل (التحميل الزائد) Overloading مع الكائنات المخصصة التي ننشئها
- كيف؟ عن طريق تعريف الدالة الخاصة المقابلة له:
 - __add__ يعمل، نعرّف + لجعل
 - __eq__ يعمل، نعرّف == لجعل
 - __len__ يعمل، نعرّف len() لجعل
 - __getitem__ يعمل، نعرّف obj[key] لجعل
- للإشارة إلى وجود الدالتين بنفس (Java/C++ مثل) أحياناً يستخدم في لغات أخرى **overloading المصطلح** الاسم لكن بمعاملات مختلفة. بايثون لا تدعم هذا بشكل مباشر، والحل البديل هو استخدام المعاملات الاختيارية. ما **Operator Overloading** نقصده هنا هو

6. Deleters: الحذف المخصص

- ، ويتم تنفيذه عند (Property) هو الجزء الثالث من الخاصية (@property.deleter) Deleter الـ: **المبدأ** على الخاصية del استخدام
- في مثال الحساب البنكي، لم نقم بوظيفته ليست مجرد حذف المتغير، بل تنفيذ منطق مخصص عند الحذف: **الغاية**
 - : بل قمنا بـoverdraft_limit فقط بحذف
 - 1. هل الرصيد سالب؟ (منع الحذف): **التحقق من شرط**
 - 2. إعادة تعيين القيمة إلى 0: **تنفيذ إجراء**
- إنه يمنحك القدرة على التحكم في دورة حياة الخاصية بشكل كامل

الفصل السابع: دفتر أخطائي (الدروس المستفادة)

- الشرط يعمل لكن الخطأ لا يحدث: الخطأ.
 - لإطلاق الخطأ ("رسالة واضحة") `raise Exception` يجب استخدام كلمة: الدرس.
- الخطأ: `TypeError: 'str' object is not callable`.
 - عند الوصول إليها () تحول الدالة إلى خاصية، لذا يجب إزالة الأقواس `@property`: الدرس.
- الخطأ: `AttributeError: 'function' object has no attribute 'setter'`.
 - `Setter` قبل الـ `@property` `Getter` يجب تعريف الـ: الدرس.
- الخطأ: الخفي عند قراءة الملفات `\n` الـ: الخطأ.
 - على السطور المقروءة لتنظيفها `.strip()` دائماً استخدم: الدرس.

الفصل الثامن : ما هي الوراثة متعددة المستويات؟

هي ببساطة سلسلة من الوراثة، مثل شجرة العائلة.

- **الجد (Device):** لديه خصائص عامة جدًا.
- **يرث من الجد، ويضيف خصائصه الخاصة (ElectricDevice): الأب**
- **يرث من الأب. وبما أن الأب ورث من الجد، فإن الابن يرث بشكل غير مباشر من الجد (PhoneDevice): الابن** أيضًا.

مثال عملي خطوة بخطوة

دعنا نبني المثال الذي ذكرته

1. أي جهاز في العالم له رقم تعريف (الجهاز) Device: **الجد**.

Python

```
class Device:
    def __init__(self, device_id):
        self.device_id = device_id
        print(f"Device with ID {self.device_id} created.")

    def get_id(self):
        return f"My ID is {self.device_id}"
```

2. هو "جهاز"، لكنه يضيف شيئًا جديدًا: يحتاج إلى كهرباء (الجهاز الكهربائي) ElectricDevice: **الأب**.

Python

```
# يرث من Device
class ElectricDevice(Device):
    def __init__(self, device_id, power_rating):
        # نستخدم id "الجهاز" لنحصل على خاصية الـ id
        super().__init__(device_id)
        self.power_rating = power_rating
        print("I am an electric device.")

    def plug_in(self):
        print("Plugging into power source...")
```

3. هو "جهاز كهربائي"، لكنه يضيف شيئًا جديدًا: يمكنه إجراء المكالمات (الهاتف) PhoneDevice: **الابن**.

Python

```
# يرث من ElectricDevice
class PhoneDevice(ElectricDevice):
    def __init__(self, device_id, power_rating, phone_number):
        # نستخدم id "الجهاز الكهربائي"
        # وهذا بدوره سيستخدم id "الجهاز"
        super().__init__(device_id, power_rating)
        self.phone_number = phone_number
        print("I am a phone.")
```

```
def make_call(self):  
    print(f"Calling from {self.phone_number}...")
```

الآن؟ (PhoneDevice) ماذا يمتلك الابن

:الآن، لننشئ كائنًا من الابن ونرى ما يمكنه فعله

Python

```
my_phone = PhoneDevice("SN123", "5V", "555-1234")
```

```
# 1. دالة من نفسه مباشرة  
my_phone.make_call()
```

```
# 2. دالة موروثية من أبيه المباشر (ElectricDevice)  
my_phone.plug_in()
```

```
# 3. دالة موروثية من جده (Device)  
print(my_phone.get_id())
```

:النتائج المتوقعة

```
Device with ID SN123 created.  
I am an electric device.  
I am a phone.  
Calling from 555-1234...  
Plugging into power source...  
My ID is SN123
```

دائمًا ما تشير إلى الأب المباشر، وهذا `super()` وورث كل شيء من سلسلة أسلافه. دالة `(PhoneDevice)` كما ترى، الابن تعمل بشكل صحيح من الجد إلى الأب ثم إلى الابن (`__init__`) ما يضمن أن سلسلة البناء

🏆 الفصل التاسع : بنك التمارين الشامل

(Easy Level) تمارين سهلة

الهدف من إتقان بناء الكلاسات والمفاهيم الأساسية: الهدف

1. `__init__` في `mass` و `name` يستقبل `planet` أنشئ كلاس: الكوكب.
2. `play`، ودالة `artist` و `title` يستقبل `Song` أنشئ كلاس: أغيتي المفضلة `by {artist}`.
3. `calculate_area`، ودالة `width` و `height` يستقبل `Rectangle` أنشئ كلاس: مستطيل بسيط المساحة.
4. `MIN_PASSWORD_LENGTH = 8` له متغير كلاس `User` أنشئ كلاس: مستخدم الموقع.
5. `Person`: لطباعة `__str__`، وقم بتعريف `age` و `name` يستقبل `Person` أنشئ كلاس: بطاقة شخصية `{name}, Age: {age}`.
6. `breed` يرث منه ويضيف خاصية `Dog` وكلاس ابن `Animal` أنشئ كلاس أب: حيوان بسيط.
7. `fuel_type` يرث منه ويعيد تعريف متغير `ElectricCar` وكلاس ابن `Car` أنشئ كلاس أب: سيارة كهربائية من "Gasoline" إلى "Electric".
8. `Number` ليجمع قيمة كائنين من `__add__`، وقم بتعريف `value` يستقبل `Number` أنشئ كلاس: جمع الأعداد.
9. `__len__`، وقم بتعريف `text` يستقبل `Word` أنشئ كلاس: طول الكلمة.
10. `(x, y)` يستقبل `from_tuple` اسمه `@classmethod` أنشئ `y` و `x` يستقبل `Point` أنشئ كلاس: صانع النقاط `Point` وينشئ منه كائن `y`.

(Medium Level) تمارين متوسطة

دمج مفهومي أو ثلاثة في كل تمرين: الهدف

1. يجب أن يكون لديه `Manager` يرث منه `Manager` وكلاس ابن `Employee` أنشئ كلاس أب: الموظف والمدير. خاصة به ودوال لإضافة وإزالة الموظفين `employees` قائمة.
2. وله دالة `Book` يحتوي على قائمة من كائنات `Library` وكلاس `Book` أنشئ كلاس: الكتاب والمكتبة. لتعيد عدد الكتب `__len__`.
3. `"Make-Model-Year"` تستقبل `from_string` اسمها `@classmethod` ودالة `Car` أنشئ كلاس: السيارة من نص `int` وتنشئ كائنًا منه مع تحويل السنة إلى رقم `Year`.
4. الخاص بها يجب أن يمنع `Setter` الـ `celsius` بخاصية `Temperature` أنشئ كلاس: درجة الحرارة الآمنة. تعيين درجة حرارة أقل من -273.15.
5. اكتب دالة خارجية تستقبل قائمة حيوانات وتستخدم `Bird` و `Fish` وأبناء `Animal` أنشئ كلاس: حقيقة الحيوان. لتجعل السمكة تسبح والطائر يطير `isinstance`.
6. `quantity` (يجب أن تكون موجبة) وخاصية `price` بخاصية `Product` أنشئ كلاس: المتجر الإلكتروني (يجب أن تكون موجبة أو صفر).
7. بناءً على `__lt__` و `__eq__` والمقارنة `(__add__)` يدعم الجمع `Vector` أنشئ كلاس: متجهات رياضية (طول المتجه).
8. للوصول لرسالة `__getitem__` يحتوي على قائمة رسائل. قم بتعريف `Mailbox` أنشئ كلاس: صندوق بريد. لطباعة عدد الرسائل `__str__` و `my_mailbox[i]` معينة عبر.

9. الخاص به يستقبل ضلعًا `__init__` يجب أن يكون `Rectangle` يرث من `Square` أنشئ كلاس: **المربع**.
لتمريره للطول والعرض في الأب `super()` واحدًا فقط ويستخدم.
10. تستقبل يوم وشهر `is_valid_date` اسمها `@staticmethod` ودالة `Date` أنشئ كلاس: **التحقق من التاريخ**.
وسنة وتتحقق مما إذا كانت صالحة (بشكل مبسط).

(Hard Level) تمارين صعبة

التعامل مع منطق معقد وتفاعل أكبر بين مكونات الكلاس: **الهدف**.

1. مع رسوم سحب. الابن يعيد `CheckingAccount` -> `BankAccount` (تمرين 3 الأصلي): **النظام البنكي**.
`super()` ليضيف الرسوم قبل استدعاء `withdraw` تعريف.
2. تعامل مع وراثة متغيرات الكلاس وإعادة تعريف `Vehicle` -> `Truck`. (تمرين 4 الأصلي): **أسطول الشاحنات**.
`@classmethod`.
3. للقراءة فقط، `area` للتحقق، خاصية `Getters/Setters` مع `Rectangle` (تمرين متقدم 3): **المستطيل الهندسي**.
لإنشاء مربع `@classmethod` للمقارنة بالمساحة، و `__eq__`.
4. التي تستخدم `overdraft_limit` مع خاصية `PremiumAccount` (تمرين متقدم 4): **الحساب البنكي المتقدم**.
بمنطق مخصص `Getter`, `Setter`, و `Deleter`.
5. `Mixins` ك `TechPowered` و `FlightEnabled` مع `SuperHero` (تمرين 5 الأصلي): **الأبطال الخارقين**.
لتطبيق الوراثة المتعددة.
6. `render` له دالة `Document` أنشئ كلاس أب: **كلاس متعدد الأشكال**.
`PdfDocument`, `WordDocument`, `HtmlDocument` كل منهم يطبق `render` دالة اكتب دالة `render_all(docs)` بطريقته.
تطبع كل أنواع المستندات.
7. `__add__` أخرى `Counter` يمكن جمعه وطرحه من كائنات `Counter` أنشئ كلاس: **عداد مخصص**.
(`__gt__`, `__lt__`, `__eq__`) ويمكن مقارنته (`__sub__`).
8. يجب أن `Card`. `Deck` يحتوي على 52 كائن `Deck` (له قيمة وشكل) وكلاس `Card` أنشئ كلاس: **بطاقات اللعب**.
ل سحب بطاقة `deal` لخلط البطاقات ودالة `shuffle` ودالة `len` يكون له.
9. للتحقق من `@staticmethod` مع دوال `DataValidator` (تمرين متقدم 3): **التحقق من الصحة المتقدم**.
تقرأ من ملف وتطبق هذا التحقق `@classmethod` للإيميل وكلمة المرور، ودالة.
10. حيث الـ `fahrenheit` و `celsius` مع خاصيتي `Temperature` (تمرين متقدم 1): **درجة الحرارة الذكية**.
الخاص بكل منهما يؤثر على الآخر `Setter`.

(Advanced Level) تمارين متقدمة

بناء أنظمة صغيرة و مترابطة، والتركيز على التصميم: **الهدف**.

1. والـ `Properties` إعادة تعريف الـ `WebsiteUser` -> `Moderator` (تمرين متقدم 2): **صلاحيات المستخدمين**.
مع استدعاء منطق الأب `Setters`.
2. يحتوي على `Inventory` كلاس `InventoryItem` -> `SellableItem`. **نظام إدارة المخزون (مبصر)**.
ت حسب القيمة الإجمالية `total_inventory_value` قاموس من هذه الكائنات وخاصية.
3. لطباعتها بشكل `__str__` أنشئ كلاسًا لتمثيل مصفوفة رياضية (قائمة من القوائم). قم بتعريف **Matrix** كلاس
لضرب مصفوفتين `__mul__` لجمع مصفوفتين، و `__add__` جميل، و

4. قم - $(2 - 5x + x^2)$ أنشئ كلاساً يمثل دالة رياضية متعددة الحدود (**Polynomial**): كلاس متعدد الحدود معيّن x لتقييم الدالة عند قيمة `__call__` لطباعتها، و `__str__` ليعتقب المعاملات، `__init__` بتعريف.
5. يحتوي على قاموس `Portfolio`. `Portfolio` (له رمز وسعر) و `Stock` أنشئ كلاس: **محفظه أسهم**. تحسب القيمة الإجمالية للمحفظة `total_value` من الأسهم وكمياتها. يجب أن يكون له خاصية أنشئ كلاساً لتمثيل الكسور (بسط ومقام). يجب أن يقوم بتبسيط الكسر تلقائياً عند إنشائه. قم **Fraction** كلاس `(__add__, __sub__, __mul__, __truediv__)` بتعريف العمليات الحسابية.
7. يمكنه إضافة موعد والتحقق `Calendar.Calendar` وكلاس `Appointment` أنشئ كلاس: **نظام حجز مواعيد** من عدم وجود تضارب في المواعيد.
8. لعرض النص الحالي `__str__` و `redo` و `undo` له دوال `TextEditor` أنشئ كلاس: **محرر نصوص بسيط**.
9. - أنشئ كلاساً يمثل آلة حالة بسيطة (مثل إشارة المرور: أخضر: **Finite State Machine**) آلة الحالة المحدودة تنتقل للحالة التالية `next_state` (أحمر). له دالة `>` - أصفر `>`.
10. الأساسي. الدوال URL يأخذ الـ `__init__` API. `requests` أنشئ كلاساً يغلف طلبات **API Client** كلاس: JSON. وتعيد البيانات بصيغة API تقوم بعمل طلبات (`get users`) (مثل

تصميم وحل مشاكل تتطلب عدة كلاسات متفاعلة بشكل معقد: الهدف

مشاريع متوسطة الحجم

1. متكامل بكلاسات OOP (المشروع الذي خططنا له) حول تطبيقك الحالي إلى تطبيق **ToDo** إعادة هيكلة تطبيق Task وTaskManager.
2. الآلة يجب أن تدير مخزونًا من المنتجات، VendingMachine وكلاس Product أنشئ كلاس: آلة بيع رقمية. تقبل "النقود"، توزع المنتج، وتعيد الباقي. يجب أن تتعامل مع حالات مثل نفاد المنتج أو عدم كفاية النقود.
3. Hotel. Hotel (لها رقم، نوع، سعر، وحالة "محجوز") وكلاس Room أنشئ كلاسات: نظام حجز فندقي بسيط. يدير قائمة من الغرف، يمكنه البحث عن غرفة متاحة، وحجز غرفة لعميل في تواريخ معينة، وإلغاء الحجز.

مشاريع كبيرة وصعبة جدًا

1. PerishableItem قم بتوسيع تمرين الخبراء. أضف أنواعًا مختلفة من المنتجات: نظام إدارة المخزون الكامل. أضف القدرة على JSON أو CSV له تاريخ انتهاء صلاحية. قم بتنفيذ حفظ وتحميل المخزون من وإلى ملفات (إنشاء تقارير مثل المنتجات التي قاربت على النفاد).
2. Student, Course, Professor, Department. يقدم مسابقات Professor. يسجل في مسابقات Student. ويحصل على درجات. النظام يجب أن يكون قادرًا على حساب معدل الطالب، عرض جدول الأستاذ، وطباعة المسابقات المتاحة في كل قسم.
3. (له Player، (لها وصف ومخرج) Room أنشئ كلاسات (كلاسيكي OOP مشروع): لعبة مغامرات نصية (take key, drop sword)، التقاط وترك الأشياء (go north) اللاعب يمكنه التنقل بين الغرف. Item مخزون وصحة)، و (unlock door)، والتفاعل مع البيئة.
4. Customer, Product, ShoppingCart, Order. أنشئ كلاسات: منصة تجارة إلكترونية (الخلفية). من Order الخاص به. عند الدفع، يتم إنشاء كائن ShoppingCart يمكنه إضافة منتجات إلى Customer. محتويات العربة وتفرغها.
5. Passenger، (له رقم، مسار، سعة مقاعد، وقائمة ركاب) Flight أنشئ كلاسات: نظام حجز طيران. النظام يجب أن يسمح بحجز راكب على رحلة، إلغاء الحجز، والتأكد من عدم حجز مقاعد أكثر من سعة. Booking. الطائرة. يجب أن يكون قادرًا على طباعة قائمة الركاب لكل رحلة.

📖 الفصل العاشر: مصادر ومراجع إضافية للتعمق

هذا الفصل هو بوابتك لتوسيع معرفتك بالبرمجة كائنية التوجه وما بعدها. التعلم لا يتوقف أبداً، والاطلاع على أساليب شرح مختلفة من مصادر متنوعة هو ما يصنع المبرمج المحترف.

1. كورسات الفيديو (Video Courses)

- **Python OOP Tutorial for Beginners - Corey Schafer:**
 - الرابط: [Corey Schafer's OOP Playlist](#)
 - المصدر الذي بدأنا به رحلتنا. يتميز بالعمق والمنهجية، ويركز على فهم "لماذا" تعمل الأشياء: الوصف: بهذه الطريقة. مثالي لبناء فهم صلب.
- **Python OOP Full Course - freeCodeCamp:**
 - الرابط: [freeCodeCamp's OOP Course](#)
 - كورس شامل ومكثف مبني على مشروع عملي. رائع لربط كل المفاهيم معاً ورؤيتها في سياق: الوصف: تطبيق واحد متكامل.
- **Python Object Oriented Programming - Tech With Tim:**
 - الرابط: [Tech With Tim's OOP Playlist](#)
 - سلسلة أخرى ممتازة تتميز بأسلوب شرح واضح ومباشر، مع الكثير من الأمثلة العملية والتمارين: الوصف:

2. التوثيق الرسمي لبايثون (Official Python Documentation)

هذا هو المصدر الأم والحقيقة المطلقة لكل ما يخص بايثون. قد يكون صعب القراءة في البداية، لكن التعود عليه مهارة لا تقدر بثمن.

- **The Python Tutorial - Chapter 9: Classes:**
 - الرابط: [Official Python Tutorial on Classes](#)
 - الشرح الرسمي من صانعي اللغة لكيفية عمل الكلاسات والكائنات وكل ما يتعلق بها: الوصف:
- **The Python Data Model (Special Methods):**
 - الرابط: [Official Python Data Model Documentation](#)
 - `__init__` مثل (Dunder Methods) هذا هو القاموس الكامل لكل الدوال الخاصة: الوصف: `__add__`, `__len__` وغيرها. عندما تنسى ماذا تفعل دالة معينة، هذا هو المكان الذي تبحث فيه. مرجع `__add__`, `__len__` أساسي للمطورين المتقدمين.

3. كتب موصى بها (Recommended Books)

عندما تشعر بالاستعداد للانتقال إلى مستوى أعمق، هذه الكتب تعتبر من أفضل ما كُتب عن بايثون والبرمجة كائنية التوجه.

- **"Fluent Python" by Luciano Ramalho:**
 - يعتبره الكثيرون أفضل كتاب للانتقال من المستوى المتوسط إلى الاحترافي في بايثون. فصوله: **الوصف** (مستوى متقدم). والدوال الخاصة بتغيير طريقة تفكيرك في اللغة (Data Model) "عن" نموذج البيانات.
- **"Python Crash Course" by Eric Matthes:**
 - فيه عملي OOP واحد من أكثر الكتب مبيعًا وشهرة لتعلم بايثون من الصفر. الجزء الخاص بالـ: **الوصف** جدًا ومبني على مشاريع، مما يجعله مناسبًا جدًا للمبتدئين.
- **"Head First Python" by Paul Barry:**
 - يشتهر بأسلوبه التعليمي الفريد الذي يعتمد على الصور والرسومات وطرق العرض غير: **الوصف** التقليدية لتسهيل فهم المفاهيم المعقدة. إذا كنت متعلمًا بصريًا، فهذا الكتاب قد يكون الأنسب لك.

BEST WISHES, THANKS,,